Design Patterns, SOLID & Git

Lecture 10 - Student Group Activities

Group number:		SE group(s):	
Student 1:		Student 2:	
Student 3:		Student 4:	
Student 5:			

Course	Software Design Patterns
Topic	Pattern review; SOLID; Git; clean code
Format	Group work - 5 students per group
Suggested time	57-65 minutes + group defense

Instructions

- Work in groups of five. Submit one completed solution per group.
- Watch or revisit the specified Lecture 10 slides before each activity.
- Discuss every answer together. Any group member may be asked to defend it.
- Give a reason linked to the scenario or code, not only a pattern or principle name.
- Complete Activity 3 and its extension in Java. Activity 5 may use pseudocode.
- Keep answers short and clear. Diagrams may be hand-drawn or digital.

Main idea: Patterns organize objects, SOLID guides dependencies, Git records changes, and refactoring improves structure while preserving behavior.

Submit: The completed worksheet, Demo.java including StaffDiscount, and the three output lines from your final run.

Activity time excludes lecture viewing and group defense. Slide numbers refer to the complete 53-slide Lecture 10 deck. The Git activity is a written exercise; no remote upload is required.

Watch first: Slides 2-22.

ACTIVITY 1	Choose a suitable design	8-10 min

For each case, choose a pattern or a simple solution. State the category where applicable and explain which part of the scenario supports your choice.

Options: Abstract Factory, Adapter, Facade, Strategy, Observer, or a simple method. Use each once.

#	Situation	Choice + category	Reason
A	Create matching buttons and menus for a light or dark theme.		
B	Your code expects print(String). An existing printer offers only printText(String), and you cannot edit it.		
C	Video conversion needs several codec and audio calls. The client wants one convertVideo operation.		
D	Checkout selects one of several discount rules while using the same total-calculation process.		
E	When a score changes, several subscribed displays must receive an update.		
F	A helper doubles an integer with one fixed rule. No alternatives are needed.		

2. How is a design pattern different from an algorithm and a design principle? Give one example of each.

3. What cost could a pattern introduce? Explain why case F does not need a Strategy class hierarchy.

Watch first: Slides 20 and 23-33.

ACTIVITY 2	Diagnose the SOLID problem	12-14 min

Name the main SOLID principle involved in each case and propose a specific repair. Other principles may also be relevant; explain your chosen main issue.

#	Design problem	Principle	Proposed repair
A	Book stores book content and also formats and prints it. Content rules and printing formats change independently.		
B	A calculator checks every operation type in one long conditional. Each new operation requires another branch in the calculation method.		
C	Account promises withdrawals. FixedTermDepositAccount inherits withdraw() but always throws an unsupported-operation error.		
D	BasicPrinter must implement print(), scan(), and fax() because one large interface requires all three. It can only print.		
E	A machine creates a StandardKeyboard inside its constructor. Changing to a gaming keyboard requires editing the machine.		

2. Does Single Responsibility mean a class must contain only one method? Explain what a reason to change means.

3. The machine now receives a StandardKeyboard through its constructor, but still declares that concrete type. Is dependency injection alone enough to give it the flexibility of a Keyboard interface?

Watch first: Slides 17-21, 25-26, and 31-32.

ACTIVITY 3	Complete and extend Java code	15 min

1. Fill every blank and save as Demo.java. Use non-negative prices in whole tenge and non-null policies. A policy must return a discount from zero up to the price.

```
interface DiscountPolicy {
  int discount(int price);
}
class NoDiscount implements DiscountPolicy {
  public int discount(int price) { return 0; }
}
class StudentDiscount implements __________ {
  public int discount(int price) { return price / 10; }
}
class Checkout {
  private final __________ policy;
  Checkout(DiscountPolicy policy) { this.policy = __________; }
  int total(int price) { return price - policy.__________(price); }
}
public class Demo {
  public static void main(String[] args) {
    Checkout regular = new Checkout(new NoDiscount());
    Checkout student = new Checkout(new __________());
    System.out.println(regular.total(10000));
    System.out.println(student.total(10000));
  }
}
```

2. Predict the first two outputs. Add StaffDiscount with a 20% discount, create a third Checkout in Demo, and print its total for 10000. Then run the program.

Policy	Predicted total	Actual total
NoDiscount		
StudentDiscount		
StaffDiscount		

3. Explain this.policy = policy. Point to the delegation call. Why can one interface type refer to different policy classes?

---

4. Which class stays unchanged when staff discounts are added? What changes to select the new policy, and how does this illustrate Open/Closed?

Watch first: Slides 34-43.

ACTIVITY 4	Manage a change with Git	10-12 min

Assume you start on an up-to-date main branch with a clean working tree. Demo.java is already tracked and origin is configured. This is a written workflow exercise.

1. Order the six commands below. Create the branch first, then edit and run Demo.java before reviewing the unstaged changes. Next stage, review staged changes, commit, and push.

```
A. git diff --staged
B. git switch -c feature/staff-discount
C. git push -u origin feature/staff-discount
D. git add Demo.java
E. git diff
F. git commit -m "Add staff discount"
```

Order: _____ _____ _____ _____ _____ _____

2. What is the difference between committing locally and pushing to origin? Does git add upload a file?

3. You stage version A of Demo.java, then edit it to version B without staging again. Which version enters a normal git commit? Explain.

4. A faulty commit is already shared on main. The team wants to undo its changes while preserving history. What does git revert provide?

5. Two branches change the same discount rule. Can Git decide which business rule is correct? What should the team do after resolving the conflict?

Extension: In GitOps, the repository requests three application instances but only one is running. What should the agent try to do? Would reverting configuration automatically restore deleted data?

Watch first: Slides 44-53.

ACTIVITY 5	Refactor without changing behavior	12-14 min

This simplified parser is inspired by Args. Object is used as a common return type for values such as an Integer or a String. Assume non-null inputs.

```
class Parser {
  Object parse(String type, String raw) {
    if ("int".equals(type)) return Integer.parseInt(raw);
    if ("text".equals(type)) return raw;
    throw new IllegalArgumentException("Unknown type");
  }
}
```

1. Replace the growing type-selection conditional with ValueParser objects. Use Object parse(String raw) as the interface method. Sketch IntParser, TextParser, and a map from a type name to a parser. Show the delegation call.

2. Add support for double in pseudocode. What new class and registration are needed? Which dispatching code should stay unchanged?

3. State the expected result or exception for each test. Preserve the behavior of the original code.

Type	Raw input	Expected result / exception
int	"42"	
text	"AITU"	
int	"twelve"	
date	"2026-11-05"	

4. Register a fake parser for int that always returns 99. What should parse("int", "42") return? What does this test prove, and what does it not prove?

Extension: A new TextParser trims spaces. The original returned text unchanged. Is that a pure refactor? Propose one test that reveals the difference.

Group Defense Preparation

When your group finishes, be ready to explain the work. The instructor may ask any one member a question.

1. Choose one Activity 1 case. What specific requirement makes your chosen pattern useful?

2. How do an algorithm, a design pattern, and a SOLID principle differ?

3. Trace student.total(10000). Which object calculates the discount, and what is printed?

4. What does this.policy = policy store? Does that line create another policy object?

5. Why does Checkout declare DiscountPolicy rather than StudentDiscount?

6. How did you add StaffDiscount? Which code stays unchanged, and which code selects it?

7. What happens if the constructor assignment to the final policy field is removed?

8. Why does a fixed-term account that rejects all withdrawals break the stated Account contract?

9. How do dependency injection and Dependency Inversion differ in Activity 2?

10. After staging a file and editing it again, which version enters a normal commit?

11. Why can a revert be useful for a bad change already shared with teammates?

12. Why does the fake-parser result 99 check delegation without proving integer parsing is correct?

Before you call the instructor:

Every member should be able to explain the pattern choices, SOLID repairs, Java output, Git workflow, and parser design. Be ready to point to the exact line or relationship behind an answer.

Check your submission: Include the completed worksheet, Demo.java with all three discount policies, and the three final output lines. The Java code must be your group's completed version. Other code may be pseudocode.

If you finish early: Complete the GitOps and whitespace extensions. Then invent one new discount policy and explain how you would test it, including a zero-price case.